Module7_lab

October 18, 2026

[1]:
```
#step 1 Load tips.csv and frame the problem
```

[2]:
```
import pandas as pd
```

[3]:
```
import sklearn
```

[4]:
```
df=pd.read_csv('Downloads/tips.csv')
```

[5]:
```
df.head
```

[5]:
```
<bound method NDFrame.head of      total_bill   tip     sex smoker   day    time
size
0          16.99  1.01  Female     No   Sun  Dinner     2
1          10.34  1.66    Male     No   Sun  Dinner     3
2          21.01  3.50    Male     No   Sun  Dinner     3
3          23.68  3.31    Male     No   Sun  Dinner     2
4          24.59  3.61  Female     No   Sun  Dinner     4
..           …    …       …      …    …      …    …
239        29.03  5.92    Male     No   Sat  Dinner     3
240        27.18  2.00  Female    Yes   Sat  Dinner     2
241        22.67  2.00    Male    Yes   Sat  Dinner     2
242        17.82  1.75    Male     No   Sat  Dinner     2
243        18.78  3.00  Female     No  Thur  Dinner     2

[244 rows x 7 columns]>
```

[6]:
```
df.describe()
```

[6]:
```
       total_bill         tip        size
count  244.000000  244.000000  244.000000
mean    19.785943    2.998279    2.569672
std      8.902412    1.383638    0.951100
min      3.070000    1.000000    1.000000
25%     13.347500    2.000000    2.000000
50%     17.795000    2.900000    2.000000
75%     24.127500    3.562500    3.000000
max     50.810000   10.000000    6.000000
```

[7]:
```
from sklearn.model_selection import train_test_split
X = df[["total_bill"]]
y = df["tip"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42)

print(f"len(X_train): {len(X_train)}")
```

```
len(X_train): 170
```

[8]:
```
# step 2 Why hold out 30%? (markdown cell)
```

By hold out 30% of our data as an unseen test set, it will test to see how well our model will work with new data rather than using data it was trained on. Testing on unseen data prevents overfitting and gives a better measure of the models training.

[9]:
```
# step 3  Fit the regression and read the slope
```

[10]:
```
from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X_train, y_train)
print(model.score(X_test, y_test))
```

```
0.4291782688312412
```

[11]:
```
print(model.coef_, model.intercept_)
```

```
[0.10889371] 0.8769576391532712
```

each extra dollar of bill adds about .11 of tip.

[12]:
```
#step 4 Draw the fitted line
```

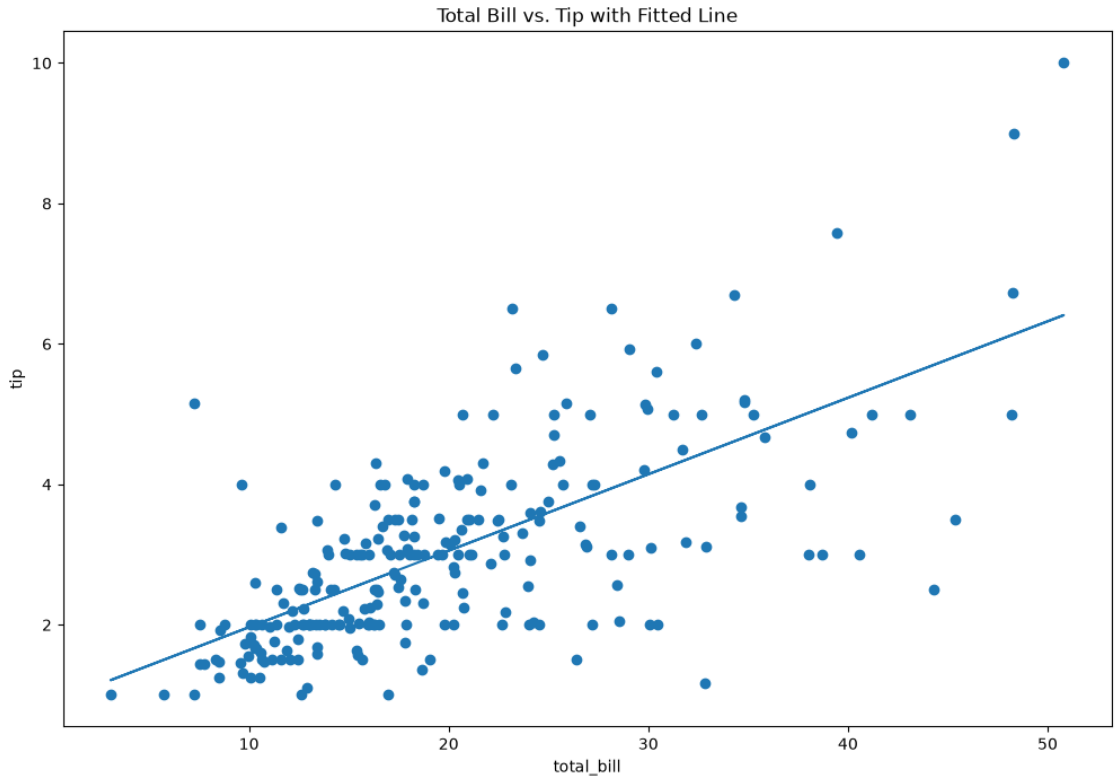
[13]:
```
import matplotlib.pyplot as plt
```

[14]:
```
y_predict=model.predict(X)
```

[15]:
```
plt.figure(figsize=(12, 8))
plt.scatter(X, y)
plt.plot(X, y_predict)


plt.xlabel('total_bill')
plt.ylabel('tip')
plt.title('Total Bill vs. Tip with Fitted Line')

plt.show()
```



[16]:
```
#step 5 load iris and split it
```

[17]:
```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
```

[18]:
```
iris = load_iris(as_frame=True)
X_iris = iris.data
y_iris = iris.target
```

[19]:
```
X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(
    X_iris, y_iris, test_size=0.3, random_state=42)
```

[20]:
```
print(f"Total flowers: {len(X_iris)}")
print(f"Training flowers: {len(X_train_i)}")
```

```
Total flowers: 150
Training flowers: 105
```

[21]:
```
#step 6 fit and score kNN
```

[22]:
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
knn = KNeighborsClassifier(n_neighbors=5)
```

```
tree = DecisionTreeClassifier(max_depth=3)
knn.fit(X_train_i, y_train_i)
print(knn.score(X_test_i, y_test_i))
```

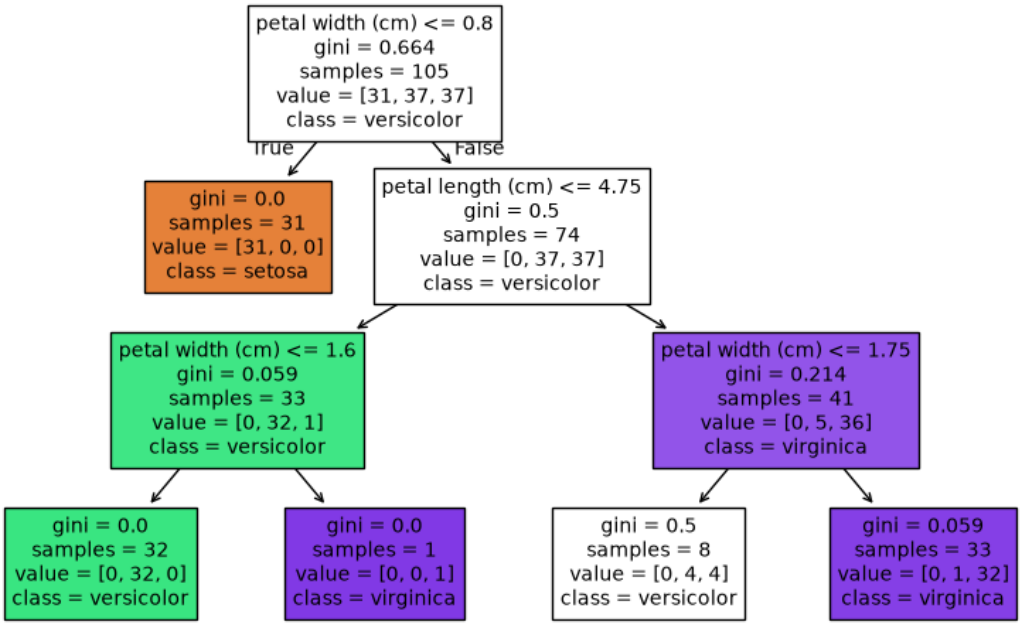
```
1.0
```

[23]:
```
# step 7 Fit, score, and read a shallow tree
```

[24]:
```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt
tree.fit(X_train_i, y_train_i)
print(f"Test score: {tree.score(X_test_i, y_test_i):.2f}")
```

```
Test score: 1.00
```

[25]:
```
plt.figure(figsize=(10, 6))
plot_tree(tree, feature_names=X_train_i.columns,
        class_names=[
        "setosa",
        "versicolor",
        "virginica",]
          ,filled=True)
plt.show()
```



Tree Diagram Meaning - The decision tree first checks whether the flower's petal length is 0.8 cm or less. If true, the flower is immediately classified as a setosa; if not, the tree moves down to evaluate other features for the remaining types.

[26]:
```
#Step 8: The overfitting probe
```

[27]:
```
from sklearn.tree import DecisionTreeClassifier
tree_deep = DecisionTreeClassifier(random_state=42)
tree_deep.fit(X_train_i, y_train_i)
print(f"{'Model':<25} | {'Train Score':<12} | {'Test Score':<12}")
print("-" * 55)
print(f"{'Shallow Tree (depth=3)':<25} | {tree.score(X_train_i, y_train_i):<12.
  ↪2f} | {tree.score(X_test_i, y_test_i):<12.2f}")
print(f"{'Deep Tree (no limit)':<25} | {tree_deep.score(X_train_i, y_train_i):
  ↪<12.2f} | {tree_deep.score(X_test_i, y_test_i):<12.2f}")
```

```
Model                     | Train Score  | Test Score
-------------------------------------------------------
Shallow Tree (depth=3)    | 0.95         | 1.00
Deep Tree (no limit)      | 1.00         | 1.00
```

- Shallow Tree at max_depth=3 shows a slight performance gap between training 0.95 and testing 1.00 scores. Meaning it'll handle a new test.
- Deep Tree at max_depth=None shows at no limit is able to get 1.00 on the training score by memorizing the training data, showing an overfitting compared to the limited model.

[28]:
```
# Step 9 - Predict one new flower
```

[29]:
```
new_flower = [[5.1, 3.5, 1.4, 0.2]]
```

[30]:
```
prediction = tree_deep.predict(new_flower)
```

```
C:\Users\user\anaconda3\Lib\site-packages\sklearn\utils\validation.py:2827:
UserWarning: X does not have valid feature names, but DecisionTreeClassifier was
fitted with feature names
  warnings.warn(
```

[31]:
```
predicted_name = iris.target_names[prediction[0]]
print(f"Predicted Class: {predicted_name}")
```

```
Predicted Class: setosa
```

[32]:
```
# Step 10: Conclusion & Summary
```

- Model Performance - Both kNN k=5 and the shallow Decision Tree of max_depth=3 shows the program was able to look at the data and tell the flowers apart
- Overfitting & Generalization - When Decision Tree max_depth=None achieved a 1.00 training score because of memorization. Since it's memorized, if new information were tested there will be issues. Thats why there needs to be limits and rules to allow for learning.
- Interpretability - Decision trees show a flowchart thats easy to follow by comparing observed values and using the charts to easily show the logic

- Failure - This model might fail if presented with rare flower with measurments outside the range or have measurement errors
- Human Application - If used on people instead of flowers, it could easily turn into a tool for stereotypes. Because the decision trees make strict yes and no or true and false based on data, the people can end up being catergorized into the wrong categories

Prompt Log AI

- How come my flower prediction is a number did I do something wrong? Also can I do the same to the Tree?

My target data was stored as a number, so i can have the iris dataset by using a target name associated to the number in this case [0]. For the Tree I had to pull the labels out of the file and then can be named

[ ]: